Description

Algorithm:
The program uses a single CUDA kernel that finds matching pairs of sample & signature, computes and stores the best average confidence score of matched pairs, and computes the Integrity Hash of matched pairs.

In particular, the program conducts a sliding-window scan over the sample for each of the pairs to find a match.

Parallelisation Strategy:

- Work Mapping: one Cuda block for each sample & signature pair → grid is 2-D with the size of (number of samples, number of signatures)
- Each block is 1-D with size of (256,1,1). The value of 256 or 512 equally contributes to high achieved occupancy of our kernel.

Theoretical Occupancy [%]	100
Theoretical Active Warps per SM [warp]	64
Achieved Occupancy [%]	99.95
Achieved Active Warps Per SM [warp]	63.97

- Within a block: threads conduct matching of signature to different portions of sample using a strided loop **for (i = threadIdx.x; i < end; i += blockDim.x).** Each thread also computes its local best confidence score and its slice of integrity hash. After the loop is completed, the kernel reduces across threads in the block to get the block's max score and total integrity hash.

Memory:

- Unified memory: All device-side data are allocated with cudaMallocManaged. The code prefetches d_samples and d_signatures to the device (cudaMemPrefetchAsync) before launching.
- Global memory access: Kernel reads samples directly from global memory through d_samples[sample_idx].
- Shared memory: Each block loads the signature from global memory to its shared memory. Two shared arrays of size block_scores and block_check_sums (int)—are used for an in-block tree reduction.
- Synchronization & atomics: In-block sync for reduction; global atomicAdd to append results without collisions.

The block size is 256 = 32 (warp size) * 4 (number of warp schedulers) * 2 (maybe more).
Rationale: allow enough resident warps to hide memory latency.

A 2-D grid is used to map each sample-signature pair to a thread block. The kernel runs with approximately 100-500 million threads and one or two million blocks.
Rationale: The smallest non-divisible unit of work is a sample-signature pair. Since threads among a block can cooperate, we allocate a sample-signature pair to a block.

Memory:

Global memory stores samples, signatures and match results. Shared memory stores an array of per-block match results for reduction.

With a block size of 256, the kernel uses 3KB of shared memory per block.

Factors affecting performance

Sequence Length:

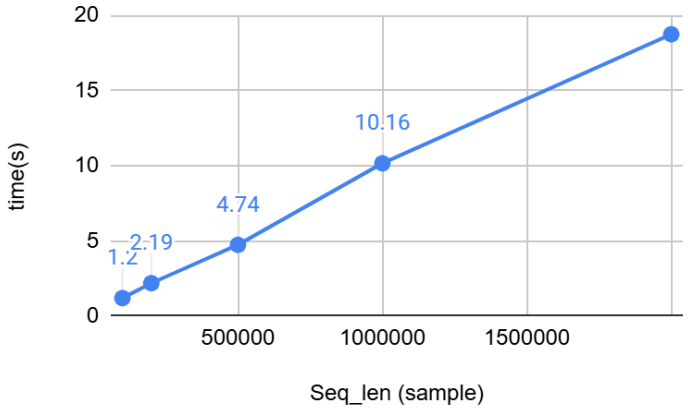


Figure 1

Observation: The program run-time scales linearly with the sample length.
Explanation:
Each block executes:

for (int i = start; i < end; i += blockDim.x)

where start = threadIdx.x and end = sample.seq_len - signature.seq_len

That loop's iteration count is roughly sample.seq_len / blockDim.x. Inside that loop, each iteration performs a fixed-cost operation that is O(1).
Since signature.seq_len is constant for the given test, the total work per block is theoretically O(sample_length), which is empirically shown in Figure 1.

Sample count:

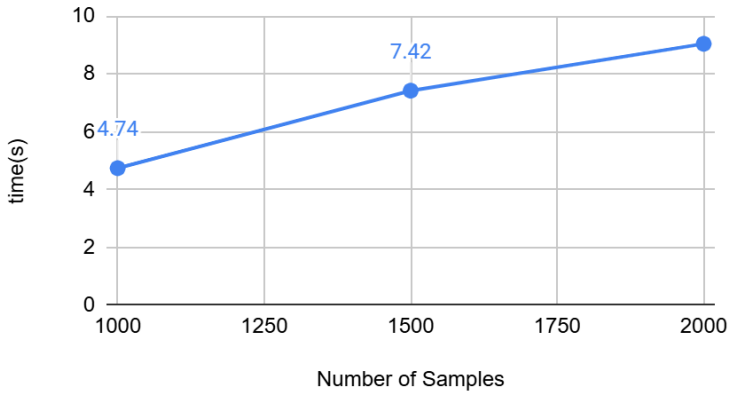


Figure 2

Observation: Program run-time increases linearly with the number of samples.

Explanation:
A100 has 42 SMs, each with 2048 threads. For a block of 256 threads, A100 can execute 336 blocks concurrently (called maximum resident blocks).

At 1000 samples and 1000 signatures, our program launches 1 million blocks which exceeds the maximum number of resident blocks. As such, each SM is scheduled with a sequence of blocks to run on (called waves per SM). In our example, waves per SM = 1,000,000 blocks / 336 = 3000 blocks.

Since waves per SM increases linearly with the number of blocks, the workload in the CUDA kernel increases linearly, explaining the trend in Figure 2.

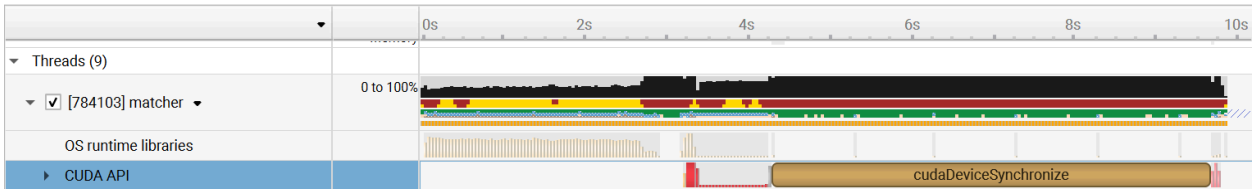
Percentage of N characters:

Run time increases by 0.1 times (10.01, 10.97) to 10 times increase in the percentage of N characters from (0.01% and 0.1%).

Existence of N characters can lead to warp divergence since a warp may not agree on execution flow. The warp stalls more often and can worsen execution time.

Optimizations

At the CUDA API level, the calls to cudaMalloc takes approximately 1s compared to cudaDeviceSynchronize which takes 5.39s. Sequential portion is 15%. We propose that this latency is acceptable and at this stage, it would be premature to hide the latency.



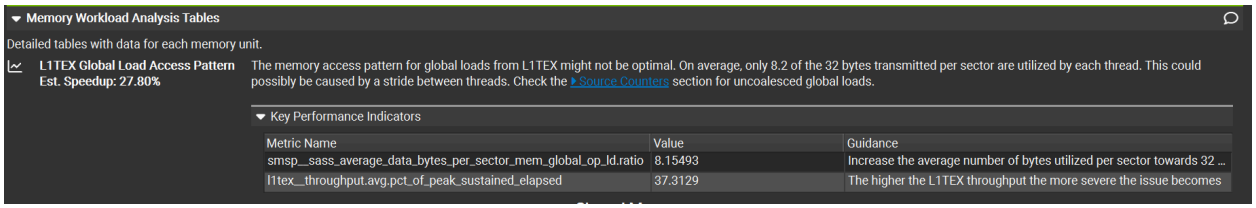
Profiling showed the program is compute-bound with high compute throughput and low memory throughput (> 10 difference).

Minimising stalls from wildcards/virus, we used an input with 0% N and virus. We detect low memory throughput and utilisation and high compute throughput. This convinces us that the program is likely compute-bound.

Optimisation 1: Using shared memory as data cache

Observing poor L2 cache hits and L1 cache pressure, use shared memory as data cache for sample sequence. Sample sequence(s) have a maximum size of 10KB, which fits in the shared memory size of 48KB (conservative estimate).

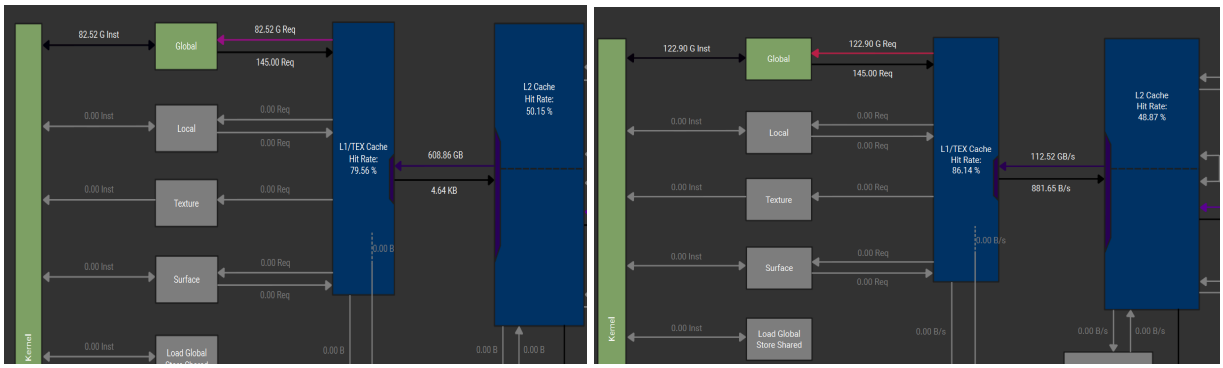
We want to find out how many cycles that our program uses for memory workload and which part of our program contributes to the memory latency.



WarpStateStatistics: we found that each warp spends ~ 15 cycles waiting for a scoreboard dependency on L1TEX making up 65% of total stalls. We hypothesized that it is due to our global memory access of d_samples and d_signatures in the kernel. As such, we attempted to bring these memory accesses to shared_memory. However, as shared_memory on A100 has a limit of 164KB per SM, we are unable to move the sample sequence as it can take up to 2GB for a sample of length 2 million.

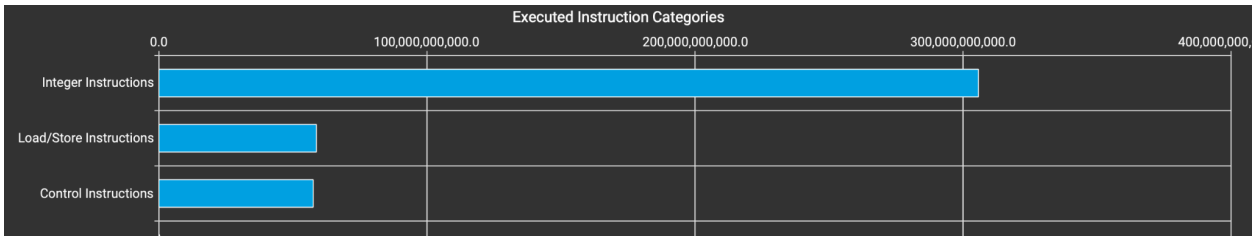
As such, only signatures are moved from global memory to shared memory.

Overall run time increased only marginally from 3.4s to 3.2s in an instance with maximum sample length. The number of global memory requests reduced from 120 G to 80 G and shared memory requests increased to (120 - 80) 40G. L2 cache hit rate increased from 48% to 50%. L1/L2 hit rates were (86.14, 48.87) and (79.56, 50.15) before and after respectively.

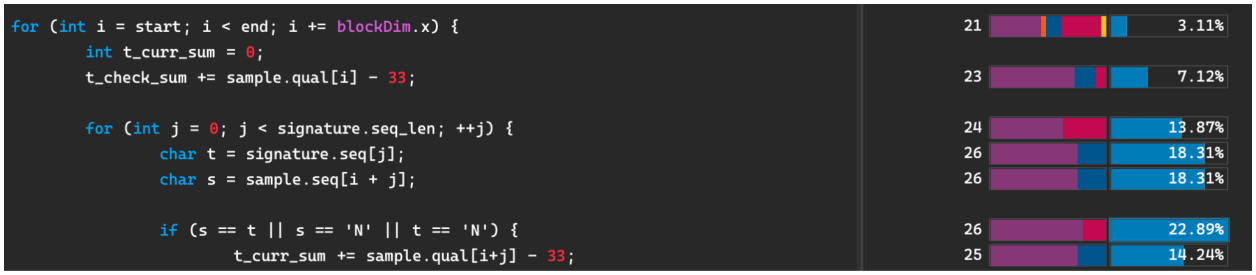


This indicates that using shared memory as data cache did not reduce the memory latency by improving access pattern. Perhaps, re-use of signatures is already present via L1 cache policies. We conclude that our optimisation attempt is not successful.

Optimisation 2: Reduce ALU hotspot



While a higher proportion of integer / ALU instructions does not warrant a program being compute-bounded, it is much higher than the other two types of instructions in our program. A deeper profiling leads us to this portion of the program where the ALU hotspot lies:



Purple is integer instruction, blue is load, red is control

Appendix

Google sheets